Granit: aprende a defender tu proyecto

Qolqe Wallet / G4
Contactos y solicitudes de dinero

Guía personal desde cero. Edición del 16 de octubre de 2026.

No necesitas empezar entendiendo todas las carpetas. Primero aprende a explicar **un caso completo**: qué debía pasar, por qué lo probaste, qué ocurrió y qué evidencia lo demuestra.

> **Tu objetivo al sustentar:**
> “El requisito exige esto. Elegí estos datos por esta técnica. Esperaba este resultado. Obtuve este otro. Esta es la evidencia y esta es mi conclusión”.

Cómo usar esta guía

Lee en orden las páginas 2 a 9 para aprender las bases. Estudia los dos casos reales en las páginas 10 y 11. Después practica preguntas y ejercicios. No memorices una frase que todavía no puedas explicar con tus palabras.

Páginas	Qué aprenderás
2-5	API, requisitos, contrato, petición y códigos HTTP.
6-9	Técnicas, Gherkin, Karate y estrategia de herramientas.
10-13	Casos reales, solicitudes, evidencias y resultados.
14-17	Carpetas, demostración, defensa, ejercicios y repaso.

Basada en el laboratorio, la Clase 10 y los archivos reales del proyecto. El énfasis en análisis y evidencia procede de lo que compartiste de la clase; no se promete una nota ni se atribuyen criterios adicionales a la docente.

Nuevo recorrido visual: páginas 18-23. Abre esas páginas si quieres ver primero los endpoints, una consulta real y Karate funcionando.

2. Lo básico: ¿qué estamos probando?

Qolqe es una billetera ficticia para practicar. Nuestro grupo revisa contactos y solicitudes de dinero, además del núcleo común: registro, login, perfil, saldo y transferencias.

Pantalla, API y base de datos

La **interfaz web** es la pantalla que usa la persona. La **API** es la puerta por la que un programa solicita operaciones. La **base de datos** guarda cuentas, contactos y solicitudes. Una acción de pantalla puede producir una petición a la API y un cambio en la base de datos.

```
Persona agrega un contacto
  -> la pantalla envía una petición HTTP
  -> la API valida y procesa
  -> la base de datos guarda, si corresponde
  -> la API responde y la pantalla informa el resultado
```

Podemos probar la API directamente, sin pulsar botones. Así enviamos entradas que la pantalla quizá impediría escribir. La API debe validar igualmente.

Palabra	Significado en nuestro trabajo
Prueba	Un experimento con contexto, acción y comprobaciones.
Esperado	Lo que debería ocurrir según la especificación.
Obtenido	Lo que ocurrió en una ejecución concreta.
Aserción	Comparación automática entre esperado y obtenido.
Defecto	Incumplimiento investigado y sustentado con evidencia.
Automatización	Código que repite peticiones y comprobaciones.

Ejemplo: esperamos crear un contacto válido. Si recibimos un rechazo, la prueba falla. Luego investigamos si falló la aplicación, el contexto o nuestra propia prueba.

Comprueba que entendiste: explica la diferencia entre “se ve bien en pantalla” y “la API aplica la regla correctamente”.

3. Aprende a leer el requisito

Una **historia de usuario (HU)** agrupa una necesidad. Un **criterio de aceptación (CA)** indica condiciones observables para aceptarla. Una **regla de negocio** establece restricciones que pueden afectar varias historias.

Identificador	Cómo leerlo
HU-G4-01	Historia 01 del grupo G4: Agregar un contacto.
CA1	Primer criterio dentro de esa historia; no es único sin indicar su HU.
R-G4-02	Regla 02 de G4: formato, longitud, normalización y unicidad del alias.
G4-004	Identificador de un caso de nuestra suite, no una HU.
D-01	Identificador de un defecto investigado.

Ejemplo central: alias

R-G4-02 exige recortar espacios exteriores; aceptar de 3 a 20 caracteres; permitir letras, incluidas tildes y ñ, números y espacios; y evitar duplicados sin distinguir mayúsculas.

De una sola regla salen varias preguntas: ¿acepta 3 caracteres?, ¿rechaza 2?, ¿permite ñ?, ¿recorta espacios?, ¿detecta ANA si ya existe Ana? No basta un único caso feliz.

No confundas las tres fuentes

Fuente	Pregunta que responde
Requisitos funcionales	¿Qué debe hacer Qolqe?
OpenAPI / contrato	¿Cómo se invoca la API y qué estructuras y respuestas declara?
Semántica HTTP	¿Qué significan los métodos y códigos en general?

> La respuesta real de la aplicación **no define el esperado**. Si copias su comportamiento como expectativa, puedes convertir un defecto en una prueba verde.

Si requisito y contrato se contradicen, conserva ambos y registra la diferencia. No elijas silenciosamente el que haga pasar el test.

4. Cómo leer una petición HTTP

HTTP es el protocolo de comunicación. Un **endpoint** se identifica por un método y una ruta. GET /api/contactos y POST /api/contactos usan la misma ruta, pero piden cosas distintas.

```
POST http://localhost:8000/api/contactos
Authorization: Bearer <TOKEN_DE_PRUEBA>
Content-Type: application/json

{
  "celular": "912345678",
  "alias": "Niño"
}
```

Ejemplo ilustrativo, no una ejecución nueva. El escenario crea sus usuarios y obtiene su token.

CRUD significa crear, leer, actualizar y eliminar. GET consulta; POST procesa una acción o crea en nuestros endpoints; PUT reemplaza en HTTP, aunque Qolqe documenta ediciones parciales; PATCH actualiza parcialmente; DELETE elimina.

Parte	Qué significa
POST	Pide procesar una creación en este endpoint.
localhost:8000	La aplicación de pruebas está en tu computadora.
/api/contactos	Recurso sobre el que se trabaja.
Authorization	Envía el token para identificar la sesión.
Content-Type	Declara que el cuerpo se envía en JSON.
JSON	Datos estructurados entre llaves: campo y valor.

Dónde viajan los datos

Ruta: /api/contactos/37 identifica un contacto. **Consulta:** ?buscar=jose filtra la lista. **Cabecera:** Authorization transporta el token. **Cuerpo:** el JSON describe los datos a crear o modificar.

JWT es el tipo de token de Qolqe. Aquí dura 30 minutos. Login obtiene el token; las peticiones protegidas lo envían. Tener token no permite usar contactos de otro usuario.

Pregunta de práctica: ¿cuál es la diferencia entre un contacto inexistente y una petición sin token? Son condiciones distintas y tienen expectativas distintas.

5. HTTP: interpretar sin repetir mitos

Código	Idea principal	Uso esperado en Qolqe
200	Éxito	Consultar o modificar con respuesta.
201	Recurso creado	Crear contacto, solicitud o transferencia.
204	Éxito sin contenido	Eliminar contacto o cerrar sesión.
401	Sin credenciales válidas	Token ausente, inválido o vencido.
404	Recurso no disponible	Inexistente o ajeno, según el proyecto.
409	Conflicto con el estado	Duplicados o transición prohibida.
422	Contenido no procesable	Validaciones y reglas inválidas.
500	Fallo inesperado	Investigar fallo del servidor.

Las distinciones que te pueden preguntar

HTTP y código de negocio: 422 es un estado HTTP; ALIAS_INVALIDO explica la causa dentro de Qolqe. Debemos comprobar ambos cuando el requisito lo exige.

201 no pertenece solo a POST. Un PUT que crea un recurso puede responder 201. **No todo dato inválido exige 400:** Qolqe usa 422 en muchas validaciones. **200 sin cuerpo no prueba por sí solo un defecto.**

403 significa que se rechaza una petición entendida; HTTP permite usar 404 para ocultar un recurso prohibido. Qolqe exige 404 para recursos ajenos. No sustituyas esa expectativa por una preferencia personal.

Caso Niño: no decimos que 422 sea un código incorrecto para rechazar datos. Decimos que este alias permitido **no debía rechazarse**. Por eso esperábamos 201.

Clase 10 trata el DELETE de Booker documentado con 201 como observación sobre su convención. Distingue cumplir un contrato de discutir su diseño. No traslades los códigos de Booker a Qolqe.

Referencia general: RFC 9110, especialmente secciones 9.3.4 y 15.3-15.5. Las reglas concretas del producto están en sus requisitos.

6. Particiones: elegir datos con razón

La **partición de equivalencia** divide entradas o situaciones en grupos que esperamos que se comporten de forma semejante. Elegimos representantes para investigar cada grupo, sin asumir que con uno detectaremos todos los defectos.

Ejemplo: celular del contacto

Situación	Esperado de negocio
Registrado, activo, ajeno y nuevo	Crear si alias y capacidad también son válidos.
Es mi propio celular	CONTACTO_PROPIO.
Formato inválido	CELULAR_INVALIDO.
No registrado o cuenta cerrada	CELULAR_NO_REGISTRADO.
Ya es mi contacto	CONTACTO_EXISTENTE.

Estas clases dependen de registro, estado y propiedad. No se resuelven probando números “cercanos”: un celular numéricamente mayor no es más activo ni más ajeno.

Ejemplo: caracteres del alias

Ana, José, Niño y A1 B representan características permitidas que interesa comprobar. Ana@ representa un carácter prohibido. La ñ merece un caso porque la regla la menciona expresamente y puede omitirse en una validación.

Aislar la condición: para probar Niño, usamos celular registrado y activo, distinto del propio, no repetido, y lista con espacio. Si también envío un celular inválido, el rechazo puede ocultar el problema del alias.

Así lo explicas

“Usé particiones porque hay situaciones con resultados de negocio diferentes. Elegí un representante por condición y mantuve válidas las demás entradas”.

Practica: ¿por qué cambiar Ana por Eva no necesariamente agrega cobertura, pero cambiar Ana por Ana@ sí investiga otra condición?

7. Límites y estados: otras dos técnicas

Análisis de valores límite

Busca errores junto a una frontera. Si la longitud válida del alias es de 3 a 20, los valores de dos puntos alrededor de cada extremo son:

Longitud	Resultado	Por qué
2	Inválido	Vecino inferior al mínimo.
3	Válido	Primer valor permitido.
20	Válido	Último valor permitido.
21	Inválido	Vecino superior al máximo.

Un alias de 10 caracteres está dentro del rango, pero no comprueba la frontera. Vacío, ausente y solo espacios pueden recorrer validaciones distintas; no los descartamos automáticamente por haber probado longitud 2.

Para máximo 15 contactos: preparar 14 y crear el 15; después intentar el 16 y comprobar rechazo sin aumentar la lista. Para máximo 5 favoritos: permitir el quinto y rechazar el sexto.

Transición de estados

Una solicitud puede pasar de PENDIENTE a ACEPTADA, RECHAZADA, CANCELADA o VENCIDA. Una acción válida depende del estado, la vigencia y quién la realiza.

```
PENDIENTE + destinatario acepta a tiempo -> ACEPTADA
PENDIENTE + emisor cancela a tiempo     -> CANCELADA
ACEPTADA  + intentar aceptar otra vez  -> rechazo
PENDIENTE + alcanza 48 horas           -> VENCIDA
```

Usamos tres puntos temporales: 47 h 59 min, 48 h y 48 h 1 min. Nuestra interpretación es vencida desde las 48 horas, coherente con la regla. Se renueva el token después de avanzar el reloj: su vigencia es de 30 minutos.

> No todas las HU necesitan todas las técnicas. Elige la técnica por la condición que debes verificar, no por incluir más nombres en el informe.

8. Gherkin y Karate, línea por línea

Gherkin estructura escenarios con Feature, Scenario y pasos. Given describe contexto; When, acción; Then, resultado. And continúa el tipo anterior. Un Scenario Outline usa una tabla Examples para repetir un comportamiento con datos distintos.

Karate usa archivos .feature y su propio DSL: instrucciones para peticiones, variables y aserciones. No es lo mismo que escribir pasos libres y enlazarlos mediante pytest-bdd.

```
@G4-004 @CA1
Scenario: Crear contacto con alias Niño
  * def r = add(b.cell, "Niño", a.token)
  * match r.statusCode == 201
  * match r.result.alias == "Niño"
  * match r.result.favorito == false
  * def list = req(
      "/api/contactos", "GET", null, a.token, {})
  * match list.result.total == 1
```

Extracto explicado del caso real; se separó una línea larga para facilitar la lectura. No es un archivo completo: los helpers y usuarios se preparan en Background.

Elemento	Qué hace
@G4-004 y @CA1	Permiten localizar el caso y su criterio; la etiqueta HU está a nivel Feature.
a / b / c	Propietario, destinatario y tercero creados para el escenario.
add(...)	Helper nuestro: envía POST /api/contactos.
r.statusCode / r.result	Nombres de nuestro adaptador: estado y cuerpo. No son los nombres nativos de Karate.
match	Aserción: exige igualdad o estructura esperada.

Karate tiene variables nativas responseStatus y response. Nuestro helper las devuelve como statusCode y result. Si una aserción falla, los pasos siguientes pueden quedar sin ejecutar.

9. Por qué estas herramientas

Elección	Por qué	Límite aceptado
Karate	Ejecuta .feature y reduce código de enlace HTTP.	Requiere Java y su DSL tiene detalles técnicos.
Schemathesis	Genera casos desde OpenAPI y comprueba conformidad.	No inventa reglas ausentes del contrato ni reemplaza el contexto de datos.
Postman / Bruno	Útiles para explorar y reproducir una petición.	No son la suite principal elegida por el grupo.
Git / GitHub	Guardan cambios, decisiones y resultados versionados.	Un historial no demuestra que domines su contenido.

Idempotencia: repetir una petición idempotente conserva el mismo efecto previsto que hacerla una vez; las respuestas pueden variar. DELETE puede dar 204 primero y 404 después. Repetir un POST de transferencia puede duplicar la operación si no existe un mecanismo que lo evite.

Cómo defender Karate frente a alternativas

Frente a Postman + Newman, priorizamos mantener escenarios ejecutables en .feature sin mantener también una colección y su correspondencia. Frente a pytest-bdd + httpx, priorizamos menos código de enlace. pytest-bdd sería fuerte si priorizáramos pasos de negocio propios en Python. Ninguna es superior para todo.

Preparación e independencia

Cada escenario crea usuarios por API; no comparte los contactos de otro caso. Background organiza ese contexto, pero no garantiza por sí solo el aislamiento. Hay que ver qué datos prepara y cómo controla el estado.

El reloj de pruebas es global: lo restablecemos antes de cada escenario y ejecutamos la suite secuencialmente. El caso de concurrencia lanza peticiones simultáneas de forma controlada dentro de su experimento; eso no significa que toda la suite sea paralela.

> Karate comprueba comportamientos elegidos; Schemathesis explora lo declarado. No diremos “ya probamos todos los campos” solo por ejecutar una herramienta.

IA: ayudó a construir y revisar los artefactos. Tu trabajo al sustentar es comprender, verificar y explicar las decisiones. No atribuyas a Franco aportes que todavía no estén verificados.

10. Caso real: Niño es rechazado

Historia: HU-G4-01. **Regla:** R-G4-02. **Caso:** G4-004. **Defecto:** D-01.

Pregunta	Respuesta
¿Qué exige?	Permitir letras, incluidas tildes y ñ.
¿Por qué Niño?	Tiene 4 caracteres y una ñ expresamente permitida.
¿Qué técnica?	Particiones de equivalencia: entrada permitida con esa característica.
¿Qué contexto?	Celular activo, ajeno y nuevo; alias único; lista vacía.
¿Qué esperaba?	HTTP 201 y contacto con alias Niño.
¿Qué obtuve?	HTTP 422 con código ALIAS_INVALIDO.

```
{
  "codigo": "ALIAS_INVALIDO",
  "detail": "El alias solo admite letras, números y espacios"
}
```

El cuerpo corresponde al rechazo registrado. Aunque el texto dice que admite letras, la validación rechaza una letra expresamente permitida por el requisito.

Conclusión que sí puedes defender

“Hay un incumplimiento: el alias debería aceptarse y la API lo rechaza. Lo fundamento en la regla y en la petición y respuesta reales”.

Conclusión que no debes inventar

La prueba se detiene cuando esperaba 201 y obtuvo 422. No afirmes que las comprobaciones posteriores de alias, favorito y total se ejecutaron en esa misma corrida. Tampoco atribuyas una causa raíz definitiva sin analizar y verificar la implementación.

Abre y explica: evidencias/G4-004.html. Identifica método, ruta, cuerpo enviado, estado recibido y código de negocio. Luego señala la regla que contradice el rechazo.

11. Caso real: Ana y los espacios

Historia: HU-G4-01. **Regla:** R-G4-02. **Caso:** G4-018. **Defecto:** D-02.

Preparación y acción

Ya existe un contacto con alias **Ana**. Intentamos agregar otro celular válido con alias **“ Ana ”**. Las comillas aquí hacen visibles los espacios exteriores; no forman parte del alias.

```
Alias existente:  "Ana"
Alias enviado:    " Ana "
Tras recortar:    "Ana"

Esperado: 409 ALIAS_DUPLICADO
Obtenido: 201; se crea otro contacto con alias Ana
```

La regla de normalización y la de unicidad deben cumplirse juntas. El resultado no cumple ambas. Que el alias devuelto esté recortado no justifica permitir un duplicado.

Comparación	Condición investigada
Ana frente a ANA	Unicidad sin distinguir mayúsculas.
Ana frente a “ Ana ”	Unicidad al aplicar normalización de espacios.
Alias con 2 frente a 3 caracteres	Frontera inferior de longitud: es otro problema.

Cómo explicarlo sin exagerar

“Elegimos otro celular para no disparar CONTACTO_EXISTENTE. Así aislamos el alias. Se obtuvo una creación cuando debía haber conflicto. La regla exige evitar ese duplicado”.

Una posible causa es comprobar duplicados antes de normalizar, pero no confundas esa hipótesis con la evidencia: la evidencia muestra la solicitud y la respuesta; la causa requiere revisar el código y verificarla.

Pregunta de la docente: “¿Por qué este caso y no otro Ana/Eva?”
Tu respuesta: “Porque aquí cambian las condiciones de unicidad tras normalización. Ana/Eva serían dos valores distintos del interior válido”.

12. Si te pide otra historia

HU de G4	Qué debes saber explicar
01 Agregar	Celular, alias y máximo 15 contactos.
02 Ver y buscar	Favoritos primero, alias, búsqueda y paginación de 10.
03 Editar	Alias sin compararse consigo mismo; máximo 5 favoritos.
04 Eliminar	Conservar transferencias; bloquear por solicitudes pendientes vigentes.
05 Transferir a contacto	Aplican reglas de HU-N-07.
06 Solicitar dinero	Monto, motivo de 3-60 caracteres, máximo 10 pendientes y SMS.
07 Ver solicitudes	Enviadas/recibidas, filtros y vencimiento.
08 Aceptar	Rol correcto, estado y plazo; transferencia; si falla, sigue pendiente.
09 Rechazar	Destinatario, estado vigente y motivo opcional hasta 60.
10 Cancelar	Solo solicitante; pendiente y no vencida.

HU-G4-08: el dinero obliga a comprobar efectos

No basta comprobar que la aceptación devuelve 200. Debe transferir el importe correcto, cambiar el estado y asociar la operación. Ante saldo o límite insuficiente, comprobar error, solicitud PENDIENTE y ausencia de cambios indebidos.

D-03 muestra una aceptación a las 48 horas cuando debía responder 409. Para evitar una falsa causa, se renueva el JWT después de adelantar el reloj.

Concurrencia: dos acciones a la vez

D-05 prueba dos transferencias de S/ 400. En el experimento de saldo, partir de S/ 500 exige que solo una se realice. En el de límite, con saldo suficiente y límite S/ 500, el uso del día tampoco debe llegar a S/ 800. Es distinto de probar dos transferencias una después de otra.

13. Cómo leer resultados y evidencia

La ejecución local registrada el **13 de octubre de 2026** tiene 116 escenarios: 110 aprobados y 6 fallos que reproducen 5 defectos. Esta guía, creada el 16, no implica que se haya vuelto a ejecutar la API hoy.

Por qué seis fallos no son seis defectos

El mismo defecto de vencimiento se detecta en dos momentos: exactamente a las 48 horas y después. Contar escenarios fallidos no equivale a contar causas o defectos distintos.

Resultado	Interpretación correcta
Caso negativo aprobado	El rechazo esperado ocurrió; no significa que se creó el recurso.
Aserción fallida	Investigar esperado, obtenido, contexto y propia prueba.
Fallo al preparar usuarios	La conducta principal puede no haberse probado.
Reporte verde	Pasaron las aserciones ejecutadas; no demuestra ausencia de defectos.
Muchas peticiones	No acredita cobertura total de HU o CA.

Qué contiene una evidencia útil

Identificador de caso, requisito, versión, momento de ejecución, petición, respuesta y comprobaciones realizadas. Los tokens se ocultan en la versión compartida. Un manifiesto vincula la ejecución con las huellas de los archivos.

El pipeline es la ejecución automática en GitHub Actions. La corrida de CI documentada tuvo 108 aprobados y 8 fallos: además de los 6 de aserción, hubo 2 de preparación. No presentamos esos dos como defectos de las HU sin investigar.

Schemathesis: revisar la señal antes de concluir

La exploración registrada generó 400 casos sobre 9 operaciones y señaló 4 fallos únicos. También advirtió falta de datos válidos. Un aviso marcó como válida una petición incompleta, pero el snapshot exige monto y motivo: rechazarla es correcto. El análisis evita reportar esa señal como bug automático.

Pendientes: variantes de permisos y destinos cerrados, filtros por estado, contenido exacto de todos los SMS y aceptación simultánea. Web, rendimiento y usabilidad no están demostrados por la suite de API.

14. Carpetas y demostración en tu PC

Ruta	Para qué la abres
docs/	Estudiar decisiones, diseño, informe, defectos y pendientes.
features/	Localizar y explicar la prueba de una HU.
features/support/	Entender usuarios, autenticación y helper HTTP.
data/	Consultar catálogo de casos y contrato original.
evidencias/	Mostrar petición y respuesta redactadas.
reports/	Ver resultados y procedencia; raw/ es local y no se versiona.
scripts/	Preparar, ejecutar o generar documentación.
apidoc/	Mostrar resultados y presentación HTML.
presentacion/	Tu guion TXT y respaldo PPT.
referencias/	Leer la aplicación recibida, no atribuirla al grupo.
.github/workflows/	Explicar GitHub Actions.
.venv/, .tools/, .build/	Dependencias y trabajo local; no estudiar todo su contenido.

Orden para la demo

1. Abre el HTML y la sección Presentación. 2. Explica HU-G4-01 y el dato elegido. 3. Abre la evidencia del caso. 4. Si te pide otra HU, usa el filtro Historias y pruebas. 5. Para una demo en vivo, prepara antes la API y ejecuta las pruebas.

Desde la carpeta del proyecto, con el entorno ya preparado:

```
# Terminal 1: API local
./.venv/Scripts/python.exe scripts/serve_api.py

# Terminal 2: suite Karate
./.venv/Scripts/python.exe scripts/run_qa.py karate
```

La suite contiene defectos conocidos: una salida distinta de cero puede ser esperada. Lee el reporte; no cambies la expectativa para obtener verde. Una evidencia guardada es una ejecución registrada, no una demo en vivo.

15. Preguntas que debes poder defender

¿Por qué Karate y no Postman?

Porque la entrega usa .feature ejecutables. Priorizamos mantener escenario y automatización juntos. Postman sigue siendo útil para explorar; no lo descartamos por incapacidad.

¿Por qué no pytest-bdd?

Karate reduce código de enlace HTTP. Aceptamos Java y pasos técnicos; pytest-bdd sería una alternativa fuerte para pasos propios en Python.

¿Por qué API y no interfaz?

Controlamos entradas y comprobamos reglas y persistencia directamente. Eso no demuestra que el flujo visual funcione: se evalúa en la entrega web.

¿Por qué 2, 3, 20 y 21?

Son los extremos permitidos y sus vecinos inválidos. Aplicamos límites de dos puntos. No afirmamos que cubran todos los defectos.

¿Por qué probar ñ?

La regla la permite expresamente y su validación puede diferir de otras letras. Mantuvimos válidas las demás condiciones.

¿Ese 422 está mal por ser 422?

No. Es un código de validación usado por Qolqe. Está mal rechazar una entrada permitida, que debía crear un contacto con 201.

¿Qué pasa si falla el login de preparación?

La conducta principal puede no ejecutarse. Se separa el fallo de preparación de un defecto del comportamiento que queríamos probar.

¿Por qué no todo está en .feature manual?

Schemathesis explora restricciones declaradas. Karate comprueba negocio y efectos. Solo delegamos cobertura técnica demostrada; revisamos lo omitido o no declarado.

Si no sabes: “No lo tengo confirmado; voy a contrastar requisito, contrato y evidencia”. Es mejor reconocer el límite que inventar una regla o causa.

16. Ejercicios y respuestas

Responde primero sin mirar la mitad inferior. Explica el porqué, no solo el número.

Ejercicio	Tu decisión
1. Alias de 2 caracteres; resto válido.	¿Qué técnica y qué código de negocio esperas?
2. Alias válido; la API responde 422.	¿Es automáticamente defecto? ¿Qué revisarías?
3. Ya existe Ana; se envía ANA.	¿Qué regla se investiga?
4. La prueba esperaba rechazo y lo obtuvo.	¿Aprueba o falla el test?
5. Token válido intenta editar contacto ajeno.	¿403 por costumbre o qué exige Qolqe?
6. Schemathesis envía peticiones y recibe muchos 404.	¿Puedes afirmar que probó todo el negocio?

Respuestas explicadas

1. Límite inferior: 2 está justo fuera del mínimo 3. Esperamos ALIAS_INVALIDO y 422, con lista sin cambios. **2.** Revisa contexto, requisito, cuerpo del rechazo y prueba; podría fallar por duplicado u otra condición. Si todo cumple y se rechaza una entrada permitida, hay incumplimiento.

3. Unicidad sin distinguir mayúsculas; esperamos ALIAS_DUPLICADO, 409. **4.** Aprueba si código, mensaje y efectos exigidos coinciden. **5.** Qolqe exige 404 para recurso ajeno. **6.** No: podría no haber llegado a lógica válida por falta de identificadores propios.

Tu prueba final de dominio

Explica G4-004 en 90 segundos sin leer. Después explica G4-018. Pide que te elijan una HU al azar y localiza su regla, técnica y evidencia. Si todavía no puedes hacerlo, vuelve al caso; no memorices más párrafos.

> Te falta comprender si solo dices “porque es mejor”, “porque dio verde” o “porque lo hizo la IA”. Sustituye cada frase por una regla, una alternativa considerada y una comprobación concreta.

17. Repaso antes de exponer

Un recorrido de estudio en cuatro sesiones

Sesión	Trabajo concreto
1 / Bases	Lee páginas 2-5 y describe una petición con tus palabras.
2 / Diseño	Lee 6-9 y justifica alias, celular y capacidad.
3 / Resultados	Lee 10-13 y abre las evidencias reales de dos defectos.
4 / Defensa	Ensaya el HTML con el guion TXT, luego sin él; responde una HU al azar.

Lista de comprobación

Debes poder explicar: requisito y CA; contrato y método; datos y técnica; esperado y obtenido; código HTTP y código de negocio; efectos; evidencia y versión; reproducción; límites y pendientes.

Antes de clase, abre los archivos y comprueba los enlaces. Usa la misma versión de referencia de las pruebas. Separa la API de prueba de servicios reales. No presentes observaciones de usabilidad como evaluadas: por ahora son un plan complementario para web.

Frase de apoyo, no para recitar

> “Elegimos este caso por esta regla y esta técnica. Mantuvimos válidas las otras condiciones. Esperábamos esto y obtuvimos esto. Aquí está la evidencia. La conclusión llega hasta lo que verificamos”.

Fuentes y dónde seguir

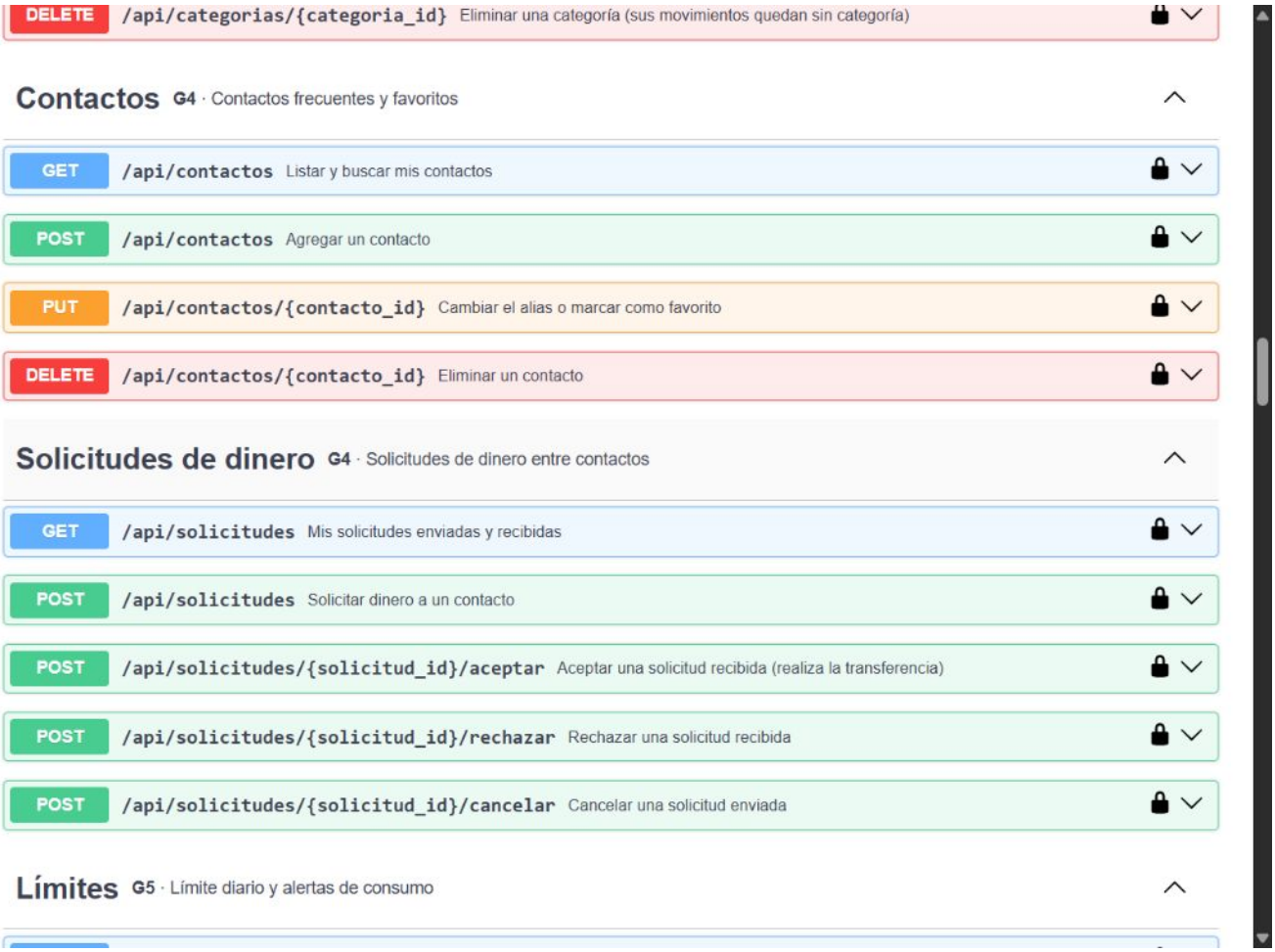
Material de clase aportado: Laboratorio 9 y Clase 10. Requisitos: referencias/qolqe-wallet/docs/requisitos.md. Referencia de aplicación: commit 8c7d2f0c939ede7765b7b80b98c10c3905693fda. Resultados: reports/resultados.json, docs/ci.md y docs/revision-contrato.md.

Repositorio del grupo: github.com/granitespinoza/qolqe-g4-qa
HTTP: RFC 9110
Karate usado: documentación de Karate 1.5.2
Schemathesis: documentación oficial

Esta guía es material personal de aprendizaje, separado del HTML que mostrarás. Está preparada con asistencia de IA y basada en artefactos verificados; comprenderlos y sustentar sus límites sigue siendo tu tarea.

18. Ver los endpoints

Captura real de entorno local de prueba / 16 de octubre de 2026



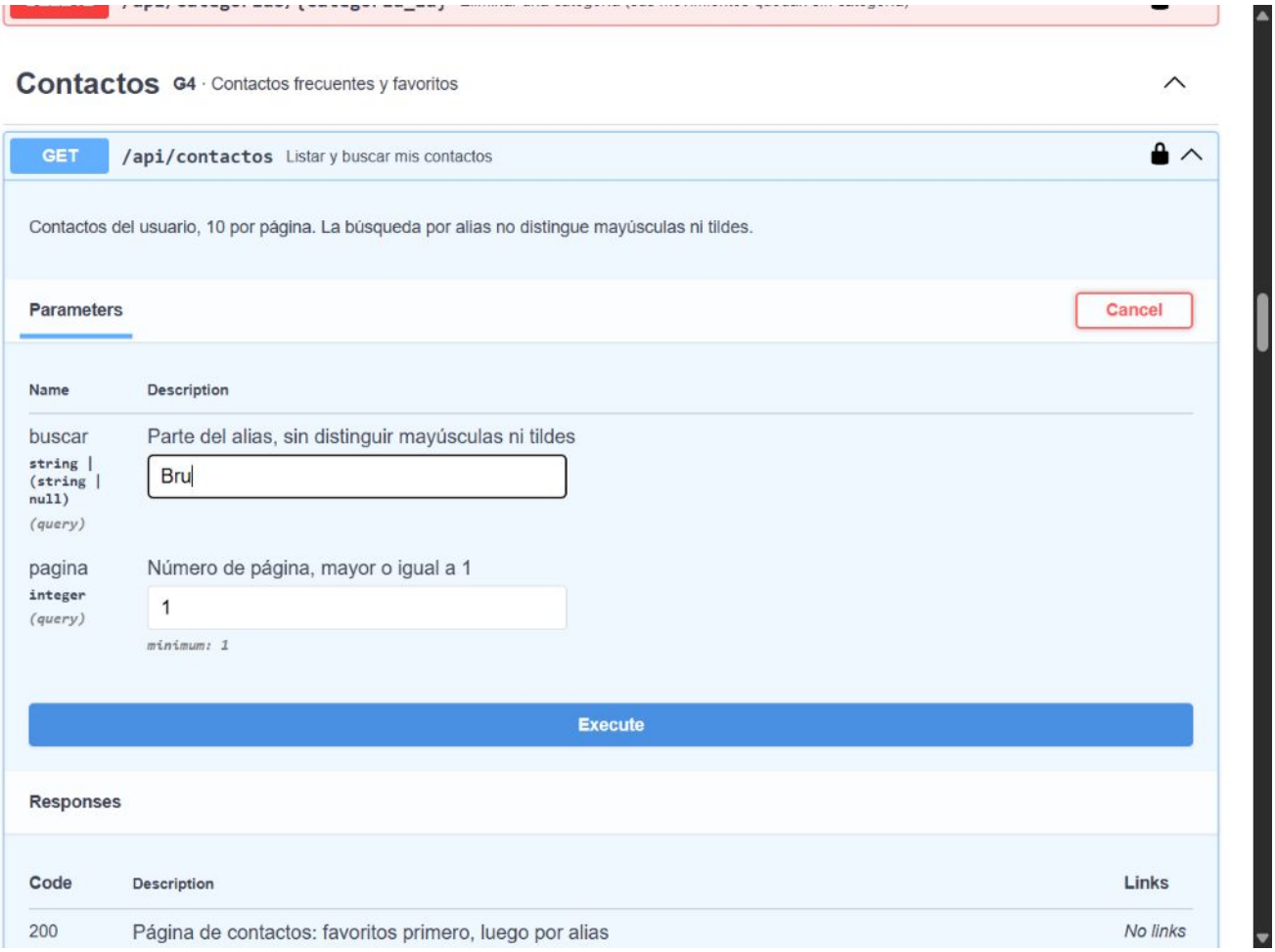
Swagger es la interfaz del contrato de la API en ejecución. GET consulta, POST crea, PUT edita y DELETE elimina en estos endpoints de contactos. El candado indica autenticación. Los colores ayudan a ubicar operaciones; no prueban que funcionen.

Cómo lo lees y qué demuestra

Abre http://127.0.0.1:8000/docs con la API encendida. Busca Contactos y abre GET /api/contactos. Swagger sirve para explorar manualmente; Karate ejecuta nuestras verificaciones repetibles.

19. Preparar una consulta

Captura real de entorno local de prueba / 16 de octubre de 2026



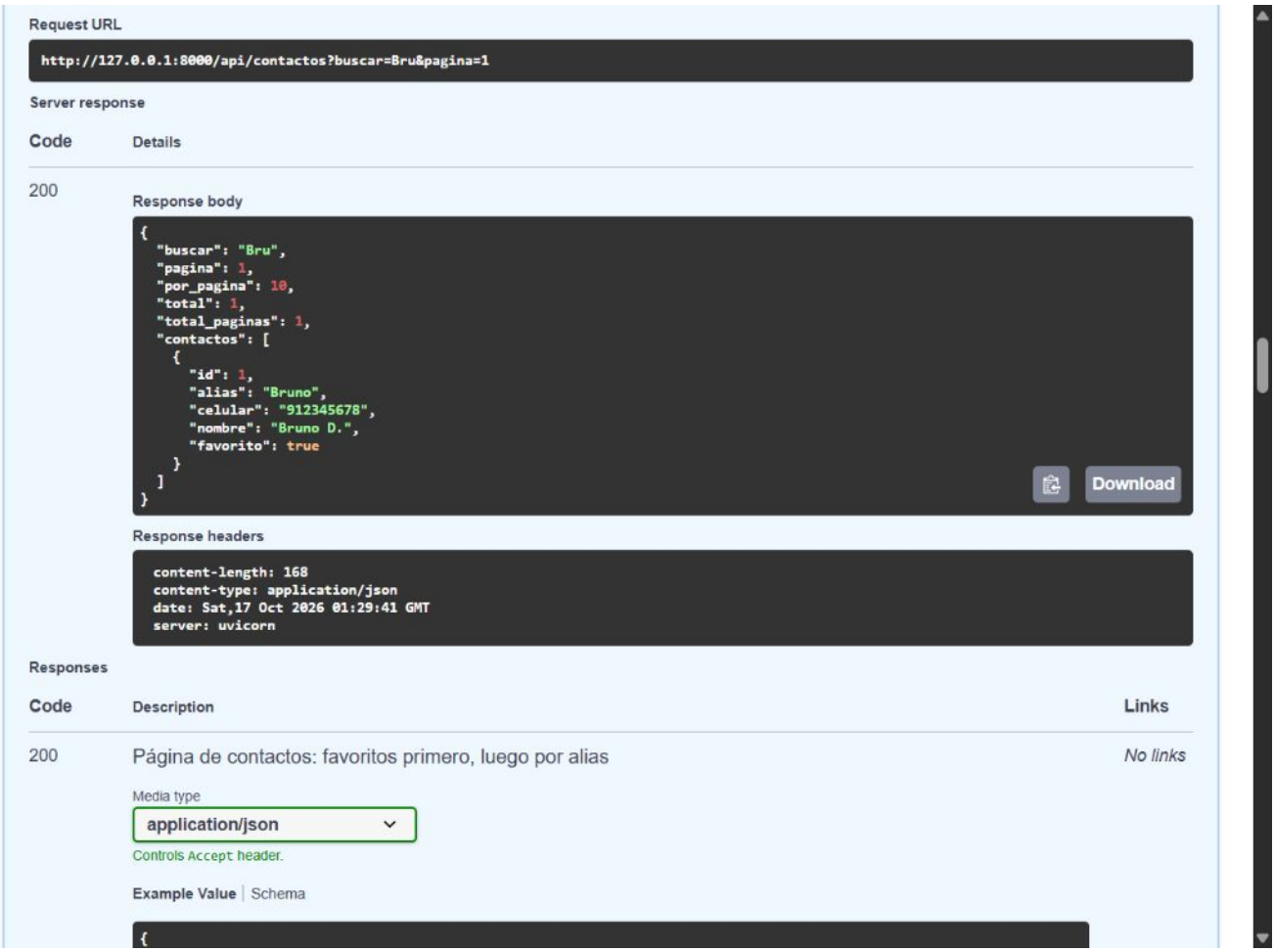
Tras iniciar sesión y usar Authorize, pulsa Try it out. En esta consulta usamos buscar=Bru y pagina=1. Son parámetros de URL; GET no necesita un cuerpo JSON para esta operación.

Cómo lo lees y qué demuestra

Antes de Execute, predice: HTTP 200 y un contacto cuyo alias contiene Bru. Esta consulta manual usa datos semilla; el escenario automatizado G4-020 crea sus propios datos y busca JOSE.

20. Leer el resultado real

Captura real de entorno local de prueba / 16 de octubre de 2026



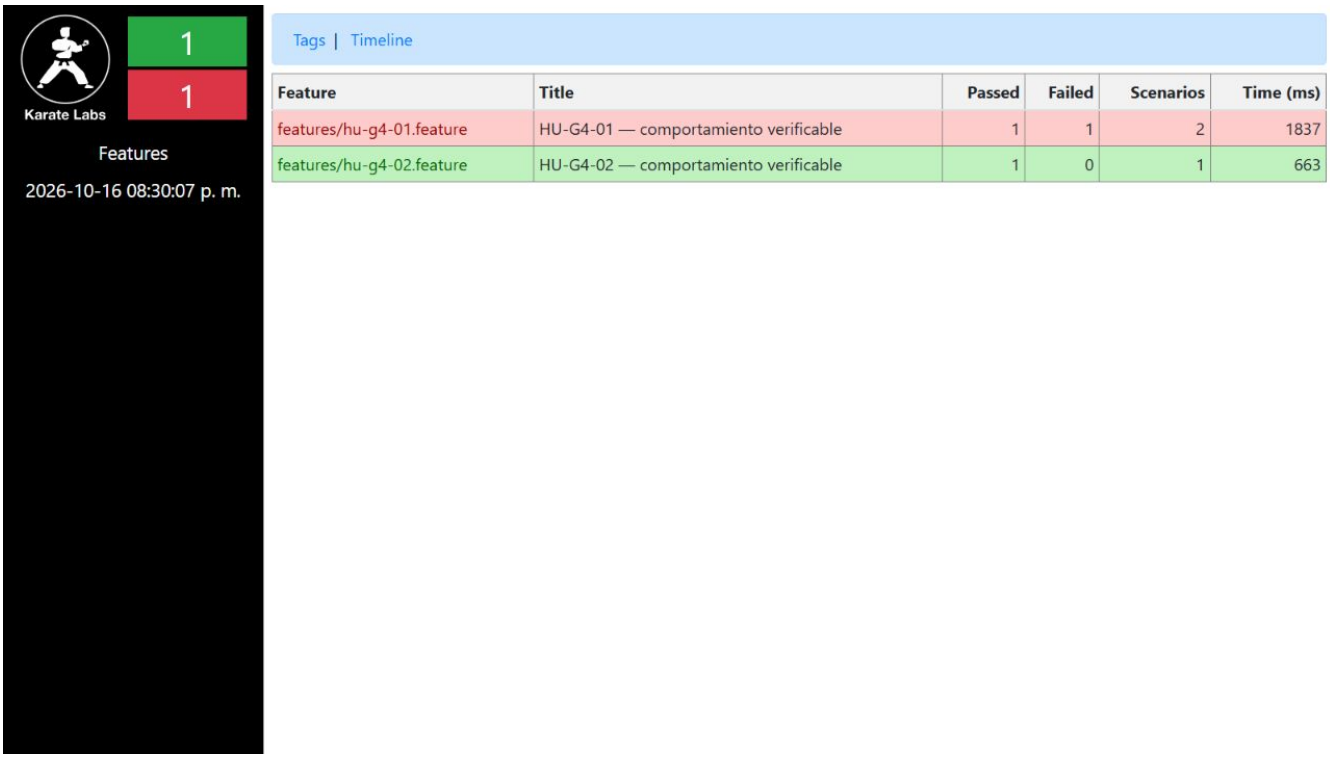
La URL identifica exactamente la consulta ejecutada. Server response muestra HTTP 200; el cuerpo devuelve total=1 y alias Bruno. Esto es una respuesta recibida, distinta del Example Value del contrato.

Cómo lo lees y qué demuestra

200 solo no basta: revisa el contenido. Aquí se comprueba la búsqueda parcial en estos datos, no todos los criterios de la HU. Vista encuadrada para excluir la cabecera de autorización; los valores de respuesta se conservan.

21. Karate sí ejecutó las pruebas

Captura real de entorno local de prueba / 16 de octubre de 2026



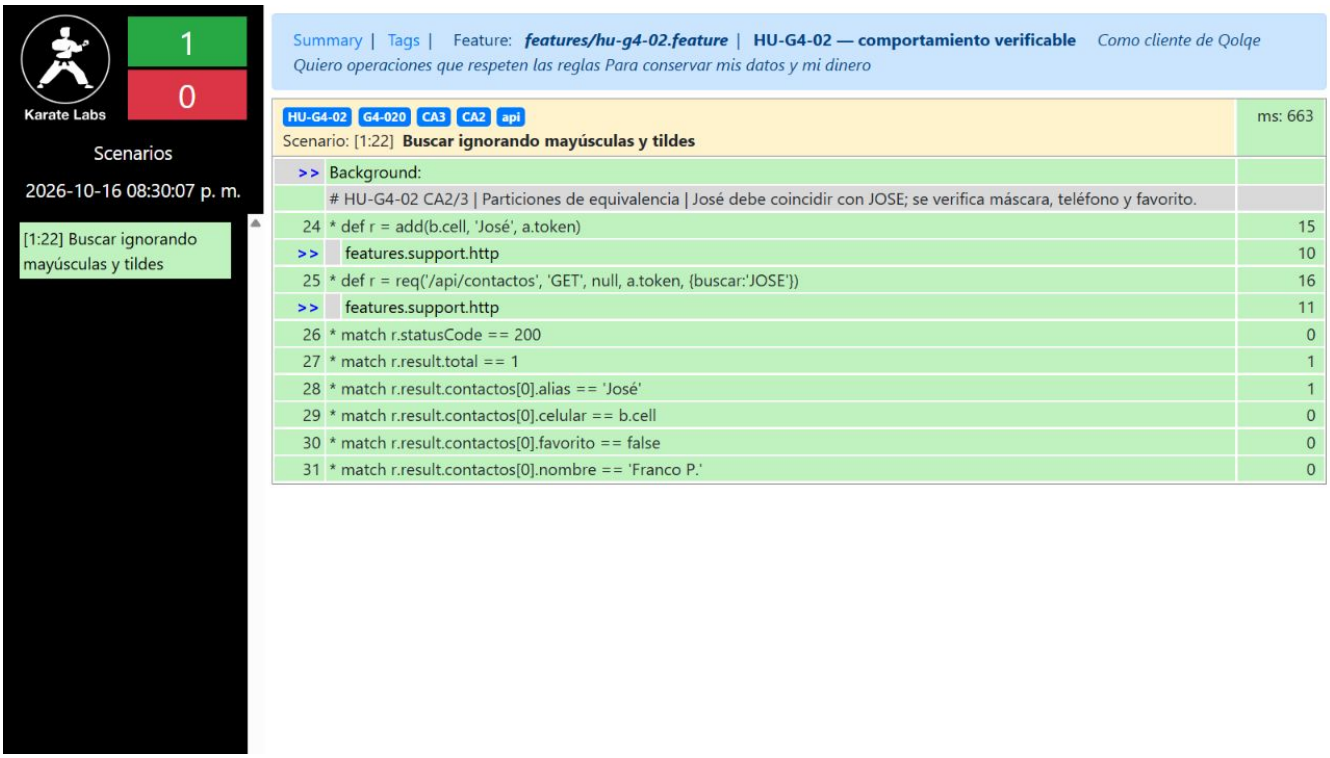
Reporte nativo de Karate 1.5.2. Demostración del 16/10/2026: tres escenarios, dos aprobados y uno fallido. HU-G4-01 aporta un aprobado y un fallo; HU-G4-02 aporta un aprobado.

Cómo lo lees y qué demuestra

Esta ejecución pequeña es independiente de la suite completa histórica de 116 casos. No sumes ambos resultados. El fallo de alias es esperado como reproducción de un defecto, pero el caso sigue figurando como fallido.

22. Leer Karate paso a paso

Captura real de entorno local de prueba / 16 de octubre de 2026



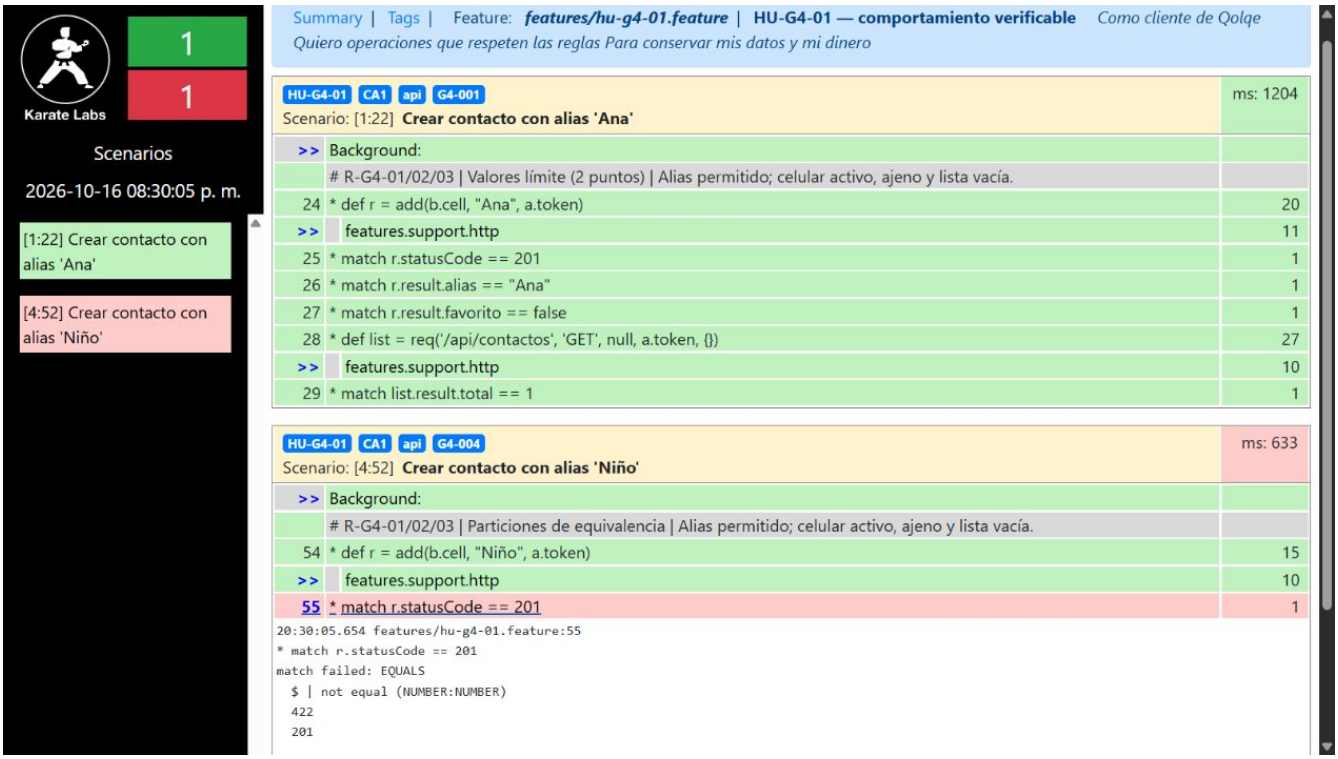
G4-020 crea José, consulta GET /api/contactos con buscar=JOSE y verifica código 200, total 1, alias, celular, favorito y nombre. Las filas verdes muestran pasos aprobados del escenario real.

Cómo lo lees y qué demuestra

def guarda valores; req llama al helper HTTP; match compara obtenido y esperado. Background prepara usuarios. Elegimos particiones de equivalencia para representar una búsqueda con diferencia de mayúsculas y tilde. Una sola entrada no demuestra todos los posibles alias.

23. Explicar un fallo con evidencia

Captura real de entorno local de prueba / 16 de octubre de 2026



G4-004 envía Niño con celular válido y usuario autenticado. La regla permite ñ: esperamos 201. Karate obtiene 422 y falla en match r.statusCode == 201. No cambiamos el esperado para volverlo verde.

Cómo lo lees y qué demuestra

Las verificaciones posteriores no se ejecutan. Para revisar el cuerpo ALIAS_INVALIDO abre evidencias/G4-004.html. Relaciona requisito R-G4-02, partición válida, petición, respuesta y defecto D-01. Así justificas la conclusión sin quedarte en un contador.